



Module 3 — Premiers tests Cypress

0 Configurer Cypress avant de commencer

Fichier à créer / modifier : `cypress.config.js` à la racine du projet

```
const { defineConfig } = require("cypress");

module.exports = defineConfig({
  e2e: {
    baseUrl: "https://example.cypress.io", // base URL utilisée par cy.visit("/")
    specPattern: "cypress/e2e/**/*.cy.js", // tous les fichiers .cy.js
    supportFile: "cypress/support/e2e.js", // fichier de support (optionnel)
  },
});
```



Explications :

- `baseUrl` : permet de mettre un **préfixe commun** pour `cy.visit()`. Exemple :

`cy.visit("/commands/actions")` // devient <https://example.cypress.io/commands/actions>

`specPattern` : indique à Cypress où **chercher les fichiers de tests**.

Même si c'est global, tu peux lancer **un fichier précis** avec `--spec` :

```
npx cypress run --spec cypress/e2e/module3-premiers-tests/exemples/visite_page.cy.js
```

`supportFile` : contient du code partagé, comme des commandes personnalisées. Au début, tu peux le laisser vide.

1

Structure des fichiers

cypress-formation/

└─ cypress/

└─ e2e/

```
└─ module3-premiers-tests/  
  └─ exemples/  
    │ └─ visite_page.cy.js  
    │   └─ assertions.cy.js  
    └─ exercices/  
      └─ exercice_visite.cy.js  
        └─ exercice_assertions.cy.js
```

2 Exemple de test avec `cy.visit()` et `baseUrl`

```
// cypress/e2e/module3-premiers-tests/exemples/visite_page.cy.js
```

```
describe("Visite d'une page", () => {  
  it("devrait charger la page Actions", () => {  
    cy.visit("/commands/actions"); // baseUrl + chemin  
    cy.contains("Actions"); // vérifie qu'un texte existe  
  });  
});
```

- Avec `baseUrl` configuré, tu n'as pas besoin de mettre `https://example.cypress.io` à chaque fois.
 - `cy.contains("Actions")` vérifie que le texte existe sur la page.
-

3 Assertions — `should` VS `expect`

◆ Rappel très important (à lire avant)

👉 Une assertion, c'est une vérification.

C'est une phrase du type :

“Je vérifie que quelque chose est vrai.”

Sans assertion :

- le test s'exécute
 - mais **il ne vérifie rien**
 - donc ce n'est **pas un vrai test**
-

a. Assertions avec `should` (pour le DOM / la page)

Phrase à retenir

`should` sert à vérifier des éléments de la page web (le DOM).

Exemple concret

```
cy.get(".action-email")
  .should("be.visible")
  .and("have.value", "");
```

Décomposition ligne par ligne

```
cy.get(".action-email")
```

- Sélectionne un élément HTML de la page
 - Ici : un champ email
 - Cypress travaille avec **le DOM** (la page affichée)
-

```
.should("be.visible")
```

- `should` = **je vérifie que...**
- `"be.visible"` signifie :

“L'élément doit être visible à l'écran”

👉 Si l'élément :

- est caché **✗**
 - n'existe pas **✗**
→ le test échoue
-

```
.and("have.value", "")
```

- `and` permet d'ajouter **une autre vérification**
- `"have.value"` vérifie la valeur du champ
- `""` signifie : **champ vide**

👉 En langage humain :

“Le champ doit être visible ET vide”

🧠 Comportement très important de `should`

👉 Cypress **attend automatiquement** que la condition soit vraie.

Exemple :

- la page met 1 seconde à charger
- Cypress **réessaie** jusqu'à ce que l'élément soit visible

💡 C'est pour ça qu'on dit que Cypress est **intelligent et asynchrone par défaut**.

Résumé `should`

- Sert à tester :
 - éléments HTML
 - visibilité
 - texte
 - attributs
 - Fonctionne avec `cy.get()`
 - Attente automatique
 - Chaînable avec `.and()`
-

b. Assertions avec `expect` (pour les valeurs JavaScript)

🧠 Phrase à retenir

`expect` sert à vérifier des valeurs JavaScript (hors page web).

Exemple concret

```
const fruits = ["pomme", "banane"];  
expect(fruits).to.have.length(2);
```

Décomposition

```
const fruits = ["pomme", "banane"];
```

- Variable JavaScript classique
- Tableau contenant 2 éléments

```
expect(fruits).toHaveLength(2);
```

- `expect` = **je m'attends à ce que...**
- `.toHaveLength(2)` signifie :
“Le tableau doit contenir 2 éléments”

👉 Si le tableau contient :

- 1 élément ❌
- 3 éléments ❌
→ le test échoue

🧠 Ce que fait (et ne fait pas) `expect`

- Teste :
 - nombres
 - chaînes
 - tableaux
 - objets
- Ne touche **pas** à la page web
- N'attend pas
- Assertion immédiate

⚠️ Erreur classique de débutant

❌ Ceci ne fonctionne pas :

```
const nombre = 42;  
nombre.should("eq", 42);
```

Pourquoi ?

- `should` appartient à Cypress

- Cypress travaille avec `cy.get()`
- Une valeur JS simple **n'est pas un élément DOM**

👉 Pour ça, on utilise `expect`.



Comparaison simple : `should` VS `expect`

<code>should</code>	<code>expect</code>
Sert à tester la page web	Sert à tester le JavaScript
Utilisé avec <code>cy.get()</code>	Utilisé avec des variables
Attente automatique	Pas d'attente
Chaînable (<code>and</code>)	Plus générique
Spécifique à Cypress	Fourni par Chai (JS)



Règle d'or

👉 Pose-toi cette question :

“Est-ce que je teste la page ou une valeur JS ?”

- Page / élément HTML → **`should`**
- Variable / calcul / tableau → **`expect`**



Exemple combiné (très courant)

```
cy.get(".action-email").should("be.visible");
```

```
const age = 18;  
expect(age).to.be.greaterThan(17);
```

👉 Test complet :

- une vérification sur la page
- une vérification sur la logique JS



À retenir absolument

1. Un test sans assertion ne sert à rien
2. `should` → DOM / page web

3. `expect` → JavaScript pur
 4. Cypress attend automatiquement avec `should`
 5. Toujours écrire des assertions lisibles
-

4 Exécution

a. Test Runner

`npx cypress open`

- Cliquez sur le fichier dans `cypress/e2e/module3-premiers-tests/exemples/`

b. Ligne de commande (fichier spécifique)

`npx cypress run --spec cypress/e2e/module3-premiers-tests/exemples/visite_page.cy.js`

5 Exercices

Exercice 1 — visite d'une page

`// cypress/e2e/module3-premiers-tests/exercices/exercice_visite.cy.js`

👉 Consigne :

1. Ouvrir la page :
2. `https://example.cypress.io/commands/actions`
3. Vérifie que le mot "**Actions**" est affiché sur la page.
4. Le test doit passer sans erreur dans le Test Runner Cypress.

Exercice 2 — assertions

`// cypress/e2e/module3-premiers-tests/exercices/exercice_assertions.cy.js`

👉 Consigne (améliorée et plus parlante)

1. Ouvrir la page **Actions**
2. Vérifier que le champ email :
 - est visible
 - est vide
3. Récupérer la valeur du champ email
4. Vérifier avec `expect` que cette valeur est bien une chaîne vide
5. Le test doit passer sans erreur